EE5203 L02 Study Sheet

Values, Decisions, Loops, and Arrays - Basri Erdogan

Essential question: How does one C loop turn eight raw sensor values into a validated average, a count, and a status decision — and how do you prove each intermediate value in the debugger?

What you should be able to do

- Trace assignment and arithmetic statements in execution order.
- Predict the result of integer division and remainder.
- Distinguish `=` (store) from `==` (compare).
- Write and trace `if`, `else if`, `else` decision chains.
- Trace `while` and `for` loops, including first and last counter values.
- Index an array safely and explain why `samples[8]` is invalid for `[8]`.
- Use the accumulator pattern to compute a sum and average.

Core statements

Statement	Question it answers
`x = 5;`	Nothing — it **stores** `5` in `x`
`x == 5`	Does `x` currently contain `5`?
`if (c) { ... }`	Run the block once **if** `c` is true
`while (c) { ... }`	Repeat **while** `c` stays true (test first)
`for (init; test; update)`	Repeat with a counter over a known range
`while (1) { ... }`	Firmware main loop — repeat forever

Operators

Group	Operators	Watch out
Arithmetic	`+ - * / %`	`10 / 3` is `3` (integer); `10 % 3` is `1`
Shorthand	`x++`, `x += 5`	Same as `x = x + 1`, `x = x + 5`
Comparison	`== != < <= > >=`	Comparing changes **neither** operand
Logical	`&& || !`	`&&` both true; `||` at least one true

The `for` loop, traced

```c
for (int i = 0; i < 4; i++) { /* body */ }
```

Repetition	`i` before body	`i < 4`?
1	0	yes
4	3	yes
stop	4	no

START creates `i = 0`, **TEST** runs before every repetition, **UPDATE** runs after every body. The body runs four times — `i` never reaches `4` inside the body.

Arrays

```c
#include <stdint.h>

uint16_t samples[8] = { 3000, 4095, 4200, 2500,     // the lecture's data set;
                        3500, 1000, 3800, 4000 };   // the lab uses its own
```

- `uint16_t`: unsigned, 16-bit, fixed-width integer — no negative values.
- Valid indices are `0` through `7`. `samples[8]` is **outside** the array.
- Loop condition is `i < 8`, never `i <= 8`.

The accumulator pattern

```
int sum = 0;                        // must start at zero

for (int i = 0; i < 8; i++) {
    if (samples[i] > 4095) {        // clamp invalid input
        samples[i] = 4095;
    }
    sum += samples[i];              // accumulate
    if (samples[i] > 2500) {
        high_count++;               // count while visiting
    }
}

int average = sum / 8;              // integer division
```

One loop can clamp, sum, and count in a single pass. Every result variable needs a correct starting value **before** the loop.

Worked example (lecture data set). Trace it yourself before reading the right column; the lab set is different, so its prediction table is still yours to fill.

Result	Value	Why
`samples[2]` after the loop	4095	`4200 > 4095` was true once
`invalid_count`	1	counted inside that same `if`
`high_count`	6	`2500 > 2500` is false — `>` is strict
`sum`	25990	running: 3000 · 7095 · 11190 · 13690 · 17190 · 18190 · 21990 · 25990
`average`	3248	`25990 / 8`; the remainder `6` is discarded
`status_code`	1	`3248 > 3000` → HIGH

Decision chains

```
if (average < 1500)        status_code = -1;   // LOW
else if (average > 3000)   status_code = 1;    // HIGH
else                       status_code = 0;    // OK
```

Conditions are tested top to bottom; exactly one branch runs; testing stops at the first true condition.

Common mistakes

- Writing `=` where `==` is meant — the condition becomes an assignment.
- Forgetting to initialize an accumulator (`sum` starts with garbage).
- `i <= 8` on an eight-element array — reads/writes outside the array.
- Forgetting the update in a `while` loop — the loop never ends.
- Expecting `25990 / 8` to have a fraction — integer division gives `3248` and discards the remainder `6`.
- Trusting a clean build as proof the computed values are correct.

Mastery self-check

- ☐ I can trace `x = x + 3; y *= 5; x++;` and give both final values.
- ☐ I can explain why `if (status = 1)` is a bug, not a comparison.
- ☐ I can state how many times `for (int i = 0; i < 8; i++)` runs its body.
- ☐ I can list the valid indices of `uint16_t samples[8]`.
- ☐ I can explain why `sum` must start at zero.
- ☐ I can predict `sum / 8` when `sum` is `25990`.
- ☐ I can place a breakpoint inside a loop and read `i` and `samples[i]`.

Practice ladder

Rung	What	When
1 Recall	this sheet and the five review questions below	before the lab
2 Guided	the L02 worksheet (solutions after the practice window)	practice window
3 Transfer	the L02 homework on CATS — new data, 10 points	after the lab, before Lab 3
4 Stretch	the LED task in the lab guide; for the project: what data will yours process?	optional

Five review questions

1. What are the final values of `a` and `b` after `int a = 6, b = 4; a = a - b; b++; a *= 2;`?
2. `int x = 7;` — what is the value of the expression `x % 3`, and what is `x` afterwards?
3. A `while` loop's body never executes. What must be true of its condition, and when was it tested?
4. In one pass over an array, which of these need a starting value before the loop: the loop counter, an accumulator, a count of high samples?
5. The lab program clamps `4200` to `4095` **before** adding it to `sum`. What would the average be wrong by if it clamped after summing?

See the L02 worksheet for extended practice. Worked solutions are released after the practice window.